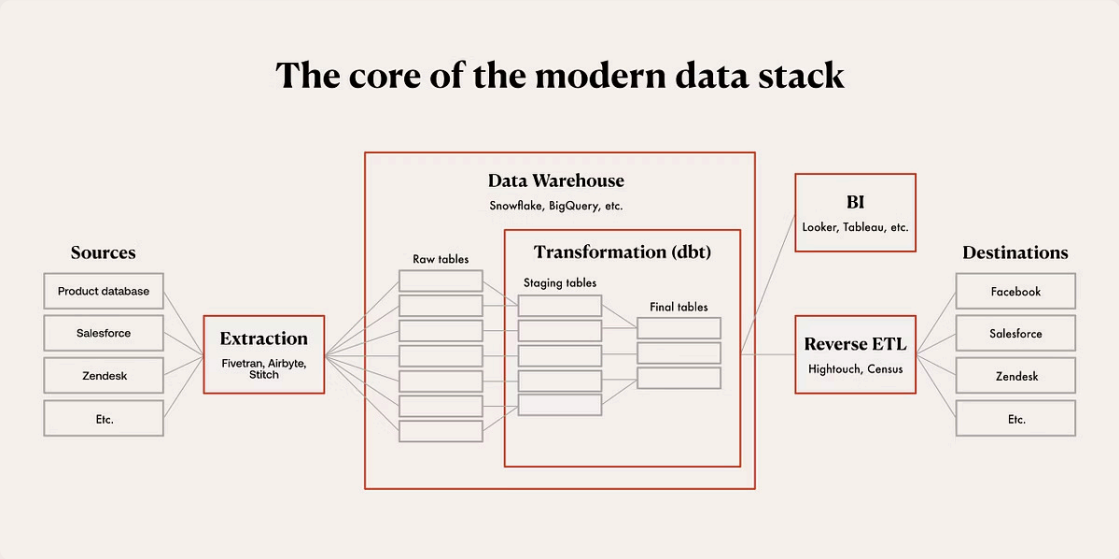
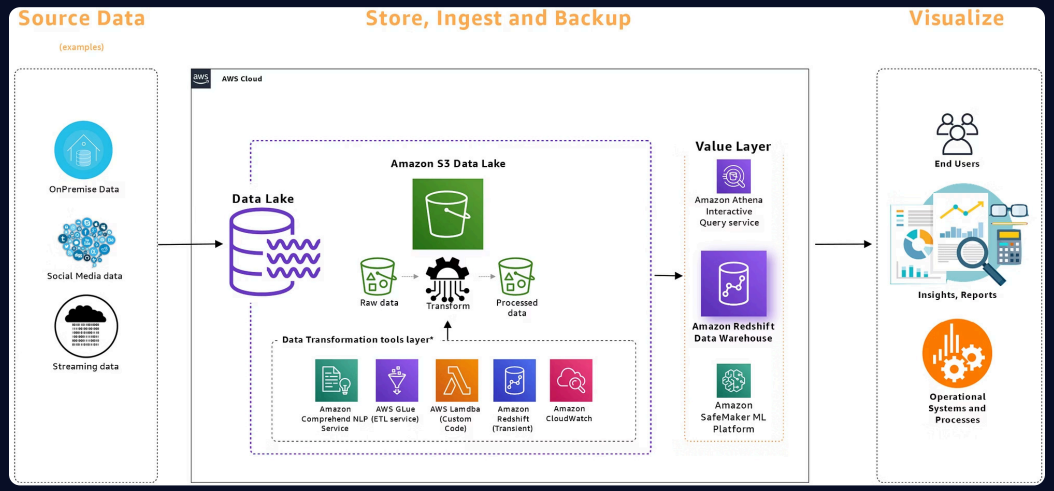


Arquitectura Moderna de Datos

¿Qué problema resuelve un Data Warehouse moderno?

Un data warehouse moderno es una **plataforma OLAP** diseñada específicamente para análisis y Business Intelligence que ofrece:

- Datos **curados, históricos y gobernados**
- **SLA** de consultas estables y predecibles
- Separación clara de los sistemas **OLTP** operacionales



En la nube incorpora ventajas adicionales:

- **Compute elástico** que escala según demanda
- **Almacenamiento económico** separado del procesamiento
- Seguridad y gobernanza gestionadas

Arquitectura de un Data Warehouse en la nube

Utilizando Snowflake como ejemplo de arquitectura moderna

Virtual Warehouses (Compute)

- Clústeres de procesamiento independientes
- Escalado vertical y horizontal
- Auto-suspensión para optimizar costes

Micro-particiones (Storage)

- Formato columnar optimizado
- Metadatos ricos para pruning
- Compresión automática

Servicios centrales

- Caché de resultados y datos
- Catálogo de metadatos
- Time Travel y Fail-safe
- Seguridad y control de acceso

Esta arquitectura separada permite escalar independientemente el almacenamiento y el procesamiento, optimizando costes mientras se mantiene el rendimiento.

Comparativa: Data Warehouse vs Data Lake vs Lakehouse

Data Warehouse	• Esquema definido y estructura rígida • Rendimiento predecible y optimizado • Ideal para métricas y reporting empresarial • Mayor coste por TB almacenado
Data Lake	• Datos crudos en object storage • Soporta datos estructurados, semi y no estructurados • Bajo coste de almacenamiento • Menor fiabilidad y rendimiento variable
Lakehouse	• Capa de **tablas ACID** sobre el lake • Combina economía del lake con fiabilidad del DW • Catálogo central para metadatos • Mejor para equipos con capacidades técnicas avanzadas

La tendencia actual es hacia arquitecturas híbridas que aprovechan lo mejor de cada enfoque según los casos de uso.

ETL vs ELT: El cambio de paradigma en la nube

ETL tradicional

Transformar **antes** de cargar:

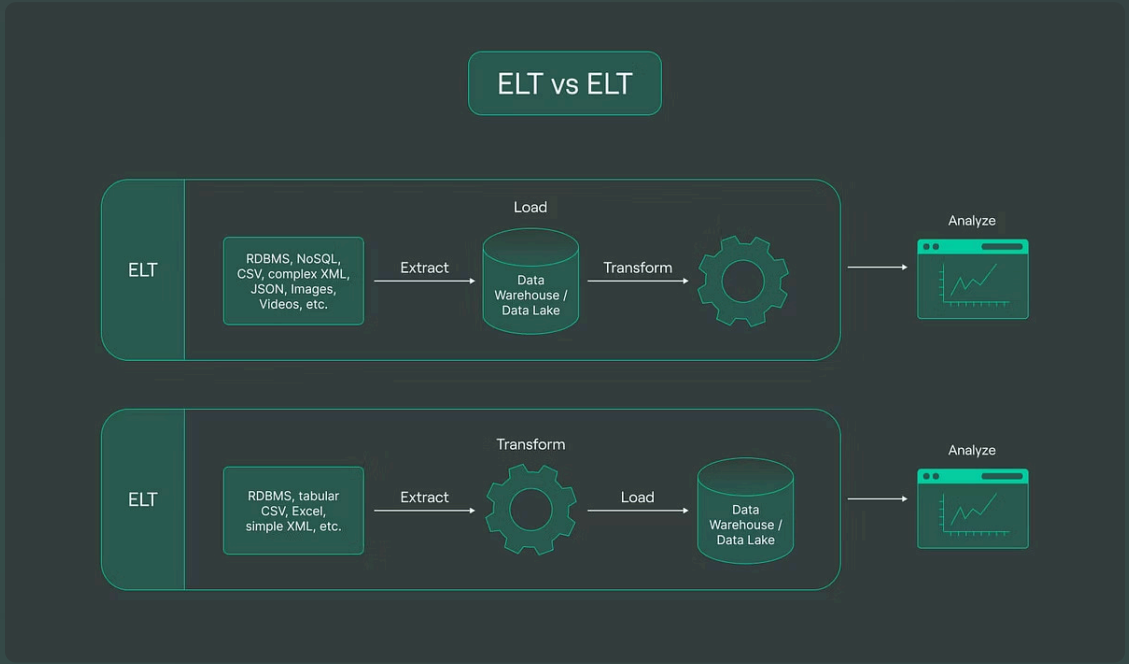
- Útil para reglas de PII previas
- Necesario con herramientas legadas
- Procesamiento en infraestructura propia
- Complejidad para escalar

ELT en la nube

Cargar crudo y transformar **dentro** del DW:

- Aprovecha compute elástico
- Utiliza SQL para transformaciones
- Patrón: **RAW → curated → marts**
- Mejor trazabilidad y gobernanza central

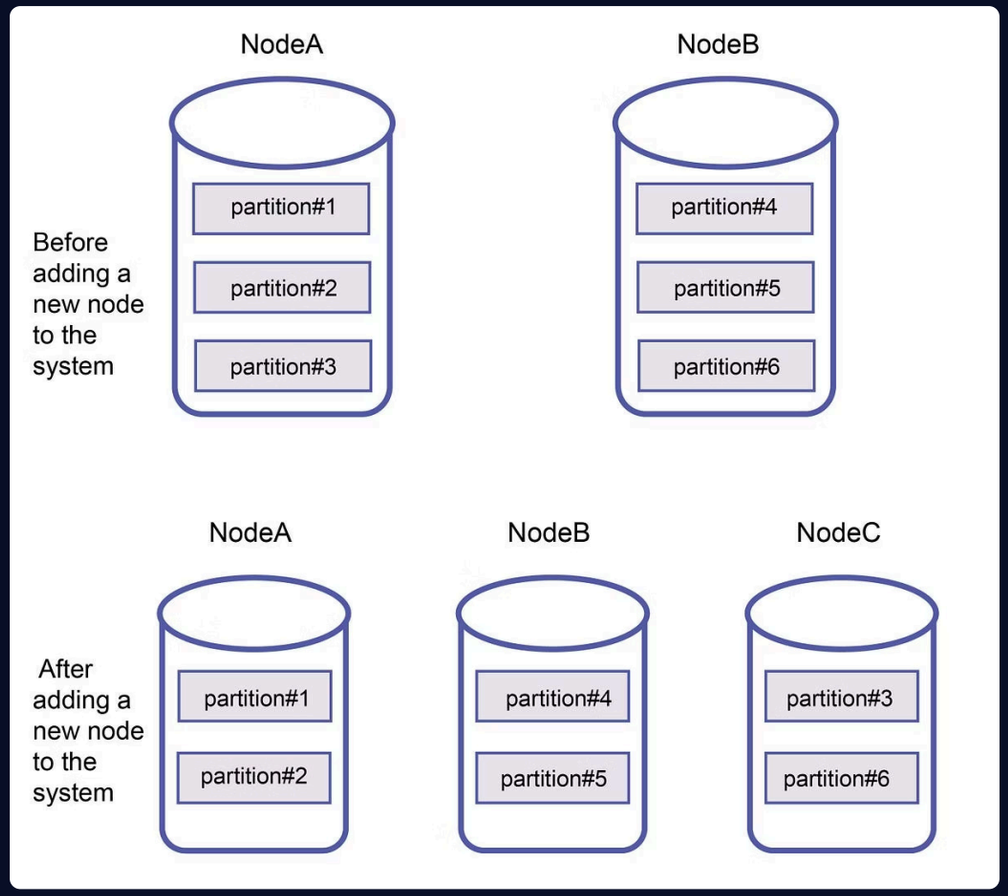
ELT gana en la nube por flexibilidad, escalabilidad y simplicidad operativa cuando los datos se procesan donde residen.



¿Qué es una partición?

Una partición es una **división física o lógica** de una tabla o dataset en **segmentos** según una **clave** determinada (p. ej., fecha, región, categoría).

El objetivo principal es **reducir drásticamente las lecturas** al permitir que el motor salte segmentos completos no relevantes para una consulta.



En data lakes

Implementado mediante estructura de carpetas/rutas (`dt=YYYY-MM-DD`)

En DW cloud (Snowflake)

Micro-particiones automáticas creadas al cargar datos (no se definen manualmente)

El particionamiento eficiente es la base de un buen rendimiento analítico, especialmente con volúmenes grandes de datos.

Tipos y prácticas de particionamiento

Por rango

Segmentación por intervalos continuos (fechas, números)

Ejemplo: un trimestre por partición

Por lista

Segmentación por categorías discretas

Ejemplo: una partición por país o región

Por hash

Distribución uniforme mediante función hash

Útil para balancear carga de trabajo

En data lakes

- Cuidar la **profundidad de carpetas**
- Controlar **tamaño de archivo** (evitar *small files*)
- Con Iceberg: **hidden partitioning** evita errores por ruta

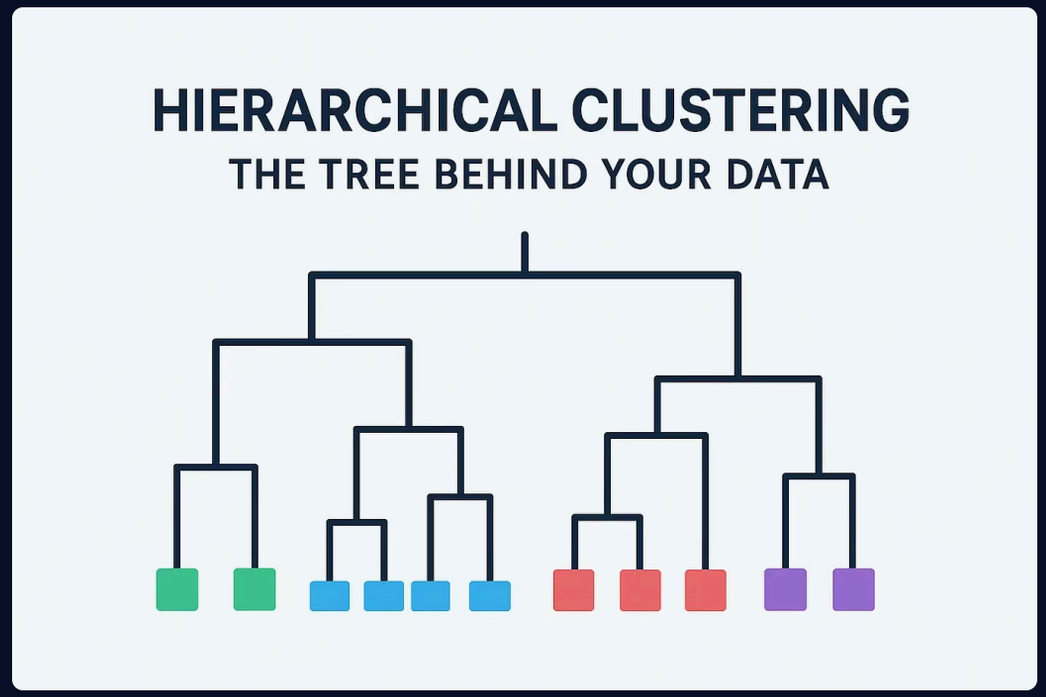
En Snowflake

- Optimizar el **orden de carga** de datos
- Cuidar la **distribución** de valores para mejor *pruning*
- Las micro-particiones se gestionan automáticamente

¿Qué es el clustering?

El clustering consiste en **ordenar físicamente** los datos de una tabla según **claves de acceso frecuentes** (p. ej., fecha, ID de cliente).

Su principal ventaja es mejorar el **data pruning**: al mantener bloques contiguos con valores cercanos, se reduce drásticamente el I/O necesario en consultas.



En Snowflake

Se implementa mediante **Clustering Keys** definidas explícitamente:

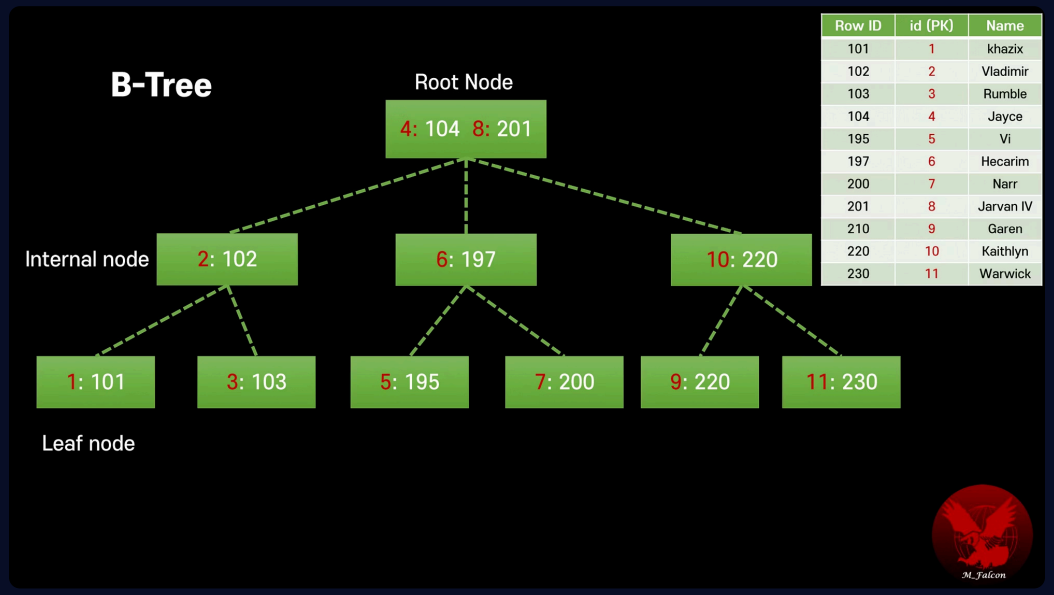
- Opcional: configurar **auto-clustering** para mantenimiento automático
- Recomendado para **tablas grandes** con patrones de filtrado repetitivos
- Evaluar el coste adicional frente a la mejora de rendimiento

El clustering es especialmente efectivo cuando el orden natural de carga no coincide con los patrones de consulta.

¿Qué es un índice?

Un índice es una **estructura auxiliar** (una especie de mapa) que acelera búsquedas muy selectivas como **point lookups** o rangos estrechos.

- En motores tradicionales: **B-tree, bitmap**
- Consumen espacio adicional
- Requieren mantenimiento (actualizaciones)



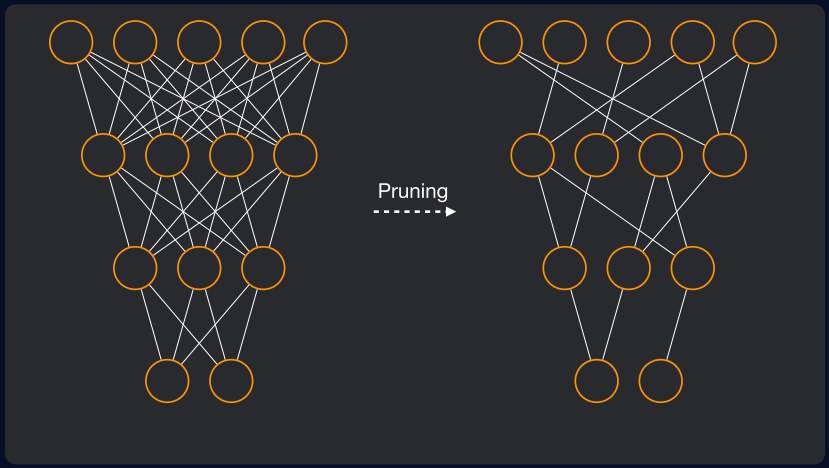
Los data warehouses cloud modernos **minimizan el uso de índices tradicionales** en favor de técnicas de particionamiento y pruning más eficientes.

En Snowflake existe el **Search Optimization Service (SOS)** específicamente para búsquedas altamente selectivas (texto, datos VARIANT, etc.).

Comparativa: Partición vs Clustering vs Índice

Partición	• Divide el **dataset** completo • Gran ahorro al **saltar** segmentos completos • Bajo coste operativo si es automático (Snowflake/Iceberg) • Primera línea de optimización
Clustering	• **Ordena** datos dentro de particiones/bloques • Mejora **pruning** en filtros comunes • Coste de mantenimiento si cambia mucho la distribución • Segunda línea de optimización
Índice	• Estructura **adicional** para localizar filas exactas • Excelente para **selectividad alta** • Añade **espacio/coste** y mantenimiento • Menos relevante en exploraciones completas

Regla práctica: Primero optimiza **particiones**; si no basta, aplica **clustering**; para búsquedas puntuales ultra selectivas, considera **índices/SOS**.



Pruning y metadatos: La clave del rendimiento

Micro-particiones en Snowflake

- Mantienen **rangos de valores** por columna
- El motor **omite** bloques completos irrelevantes
- Reducción drástica de I/O sin intervención manual

Manifiestos en Iceberg/Delta

- **Manifiestos/metadata** permiten **data skipping** a escala
- Catálogo centralizado de metadatos
- Estadísticas que guían el plan de consulta

El pruning eficiente es el factor más importante para reducir la cantidad de datos escaneados, lo que disminuye tanto el tiempo de consulta como los costes de procesamiento.

Diseño físico: Cuándo utilizar clustering

Úsalo cuando tengas una tabla muy grande

El clustering es especialmente útil para tablas de varios TB con millones o miles de millones de filas donde el coste de mantenimiento se compensa con la mejora de rendimiento.

Filtros recurrentes por pocas columnas

Identifica 1-3 columnas que aparecen frecuentemente en cláusulas WHERE con alta selectividad. Estas son candidatas ideales para clustering keys.

El orden natural de carga no alinea con tus filtros

Si tus datos llegan ordenados por timestamp pero consultas principalmente por región o cliente, el clustering puede reorganizar físicamente los datos para optimizar esos accesos.

Recomendación: Revisa la **selectividad** de tus columnas y el **Query Profile** antes y después de implementar clustering. Evalúa el **auto-clustering** y su coste asociado.

"Indexación" efectiva en Data Warehouses cloud

Evita pensar en índices clásicos como solución universal. Los DW modernos utilizan enfoques diferentes:

- **SOS (Snowflake)** para búsquedas por valores raros o patrones de texto/JSON
- No acelera escaneos completos ni agregaciones
- Coste adicional que debe justificarse

Alternativas más efectivas:

- **Vistas materializadas** para agregados frecuentes
- **Colocar datos** donde el pruning sea máximo
- Diseño orientado a patrones de consulta

Optimización de lectura: Prácticas principales

Evita SELECT *

Utiliza **proyección mínima** seleccionando solo las columnas necesarias. Reducirás I/O y mejorarás rendimiento, especialmente en columnas anchas.

Filtra temprano

Aplica filtros por columnas con buen pruning (fechas/keys) lo antes posible en la consulta. El orden de las cláusulas WHERE importa en algunos motores.

Aprovecha cachés

Utiliza **result cache** y **data cache** para consultas repetitivas. Considera **vistas materializadas** para reportes pesados ejecutados regularmente.

El rendimiento óptimo se logra combinando un buen diseño físico con patrones de consulta eficientes. Utiliza el **Query Profile** para identificar cuellos de botella específicos en tus consultas más costosas.

Optimización de compute: Balance coste vs latencia

Dimensionamiento

- Ajusta warehouses al patrón de uso real
- Activa **auto-suspend/resume** para evitar pagar por ociosidad
- Configura **multi-cluster** solo si hay colas de consultas
- Prueba **Query Acceleration** para escaneos pesados puntuales

Métricas clave

- Mide **coste por consulta** no solo tiempo total
- Evalúa **tiempo de pared** al cambiar tamaño
- Considera separar warehouses por tipo de carga (ETL vs BI)
- Monitoriza uso de recursos por query y usuario

Estrategias de ingesta de datos

Batch

Cargas programadas periódicas mediante stages y COPY INTO

- Para volúmenes grandes con ventanas de procesamiento definidas
- Máxima eficiencia de recursos
- Latencia de horas a días

Micro-batch / Near real-time

Servicios automatizados como Snowpipe

- Detecta nuevos archivos y los carga automáticamente
- Latencia de minutos
- Sin gestión manual de jobs

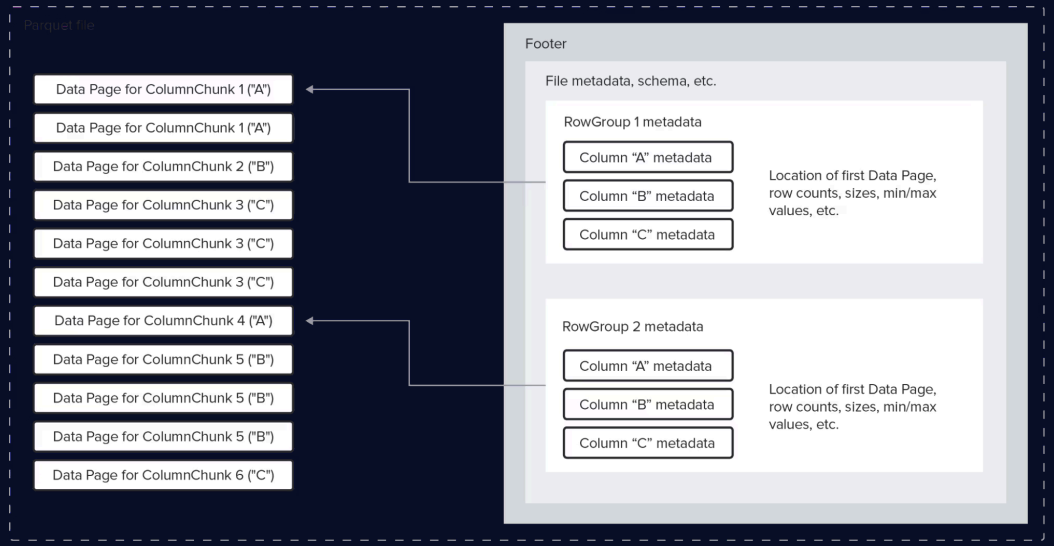
Streaming

Snowpipe Streaming/SDK para latencia baja

- Ideal para CDC (Change Data Capture)
- Latencia de segundos
- Mayor complejidad y coste

La estrategia óptima depende del balance entre frescura de datos necesaria, volumen, complejidad operativa y presupuesto disponible.

Formatos de archivo columnar



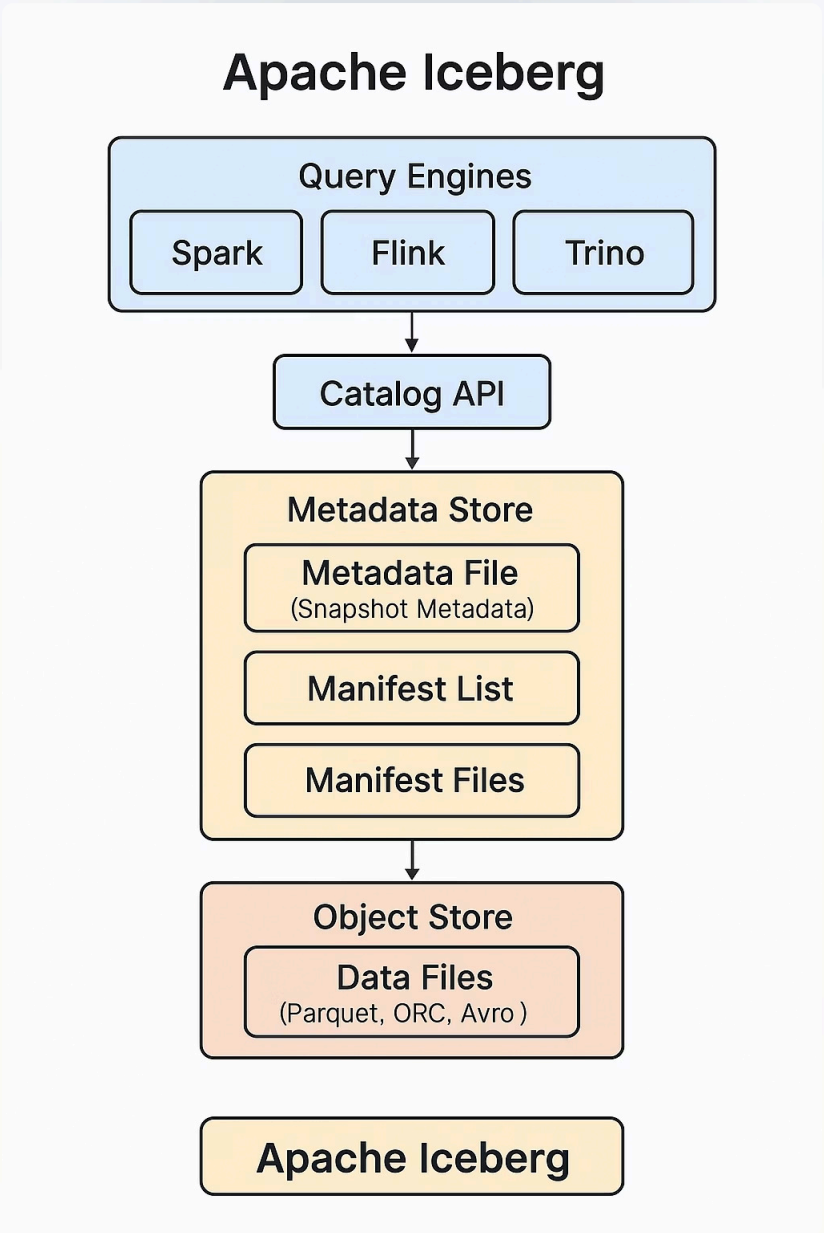
Parquet

- Organización columnar eficiente
- Compresión y encodings optimizados
- **Predicate pushdown** para reducir I/O
- Amplio soporte en ecosistema big data

Alternativas

- **ORC:** Alta compresión y estadísticas ricas (ecosistema Hive)
- **Avro:** Esquema explícito, útil para streaming/serialización

Los formatos columnares son fundamentales para el rendimiento analítico al leer solo las columnas necesarias y permitir compresión eficiente por tipo de dato.



Formatos de tabla: Apache Iceberg

Características principales

- Transacciones **ACID** en data lakes
- **Time travel** y viajes en el tiempo
- Evolución de esquema y partición sin migración
- **Hidden partitioning** independiente de rutas físicas

Ventajas operativas

- **Catálogo** unificado entre motores (Spark/Trino/Flink)
- Compatible con **Snowflake Iceberg Tables**
- **Compaction** para controlar small files
- **Data skipping** mejorado con metadatos avanzados

Iceberg representa la evolución de los data lakes hacia mayor fiabilidad y rendimiento, acercándolos a las capacidades tradicionales de los data warehouses.

Optimizando costes y evitando trampas comunes

Gestión de Time Travel y Fail-safe

Estas funcionalidades incrementan el almacenamiento si no gestionas adecuadamente la retención. Configura periodos de retención adaptados a tus necesidades reales de recuperación.

Warehouses activos sin uso real

Configura siempre auto-suspend con tiempos agresivos (5-10 minutos) y monitoriza patrones de actividad para ajustar tamaños y número de clusters.

Monitorización y observabilidad

Implementa revisiones regulares de Query History/Profile, métricas por warehouse, tablas "calientes" y sesgos de valores para optimizar continuamente.

Recomendación final: Comienza con tablas RAW que incluyan metadatos de ventana y run_id. Verifica la idempotencia para poder reprocesar sin duplicar. Aprovecha micro-partitions y pruning, evalúa clustering solo cuando sea necesario, y utiliza Streams/Tasks para procesos incrementales.